Simulator 2: Code Generation with Claude Code

Alignment: Domains 3 and 5 · Runs on the Claude Agent SDK through your Claude login, against mock backends

Objective

Prove, with Claude Code itself, how a team workspace behaves: the project `CLAUDE.md` and its `@import` files load on every call, a path-scoped rule loads only when the matching file is touched, the shared `/review` command and the forked `codebase-map` skill run non-interactively, a 47-file migration stays in plan mode while a one-file bug is fixed directly, and `--resume` and `--fork-session` branch a named investigation.

Scenario (official exam scenario 2)

"You are using Claude Code to accelerate software development. Your team uses it for code generation, refactoring, debugging, and documentation. You need to integrate it into your development workflow with custom slash commands, CLAUDE.md configurations, and understand when to use plan mode vs direct execution."

Primary domains: Claude Code Configuration & Workflows, Context Management & Reliability.

Prerequisites

- `uv` and a Claude login (`claude auth login`). Run `uv sync` once in the `labs` folder. No API key.
- Open this simulator in your browser with `uv run serve.py` from `labs`, or run the commands below from this simulator's folder.

© 2026 Tertiary Infotech Academy · UEN 201200696W

What is in the folder

Path	Role
`workspace/CLAUDE.md`	Universal standards plus two `@import` lines (`docs/standards/*.md`)
`workspace/.claude/rules/*.md`	Four rules with YAML `paths:` globs, including `**/*.test.*`
`workspace/.claude/commands/review.md`	Project-scoped `/review` with `argument-hint` and `allowed-tools`
`workspace/.claude/skills/codebase-map/SKILL.md`	Skill with `context: fork`, read-only `allowed-tools`, `argument-hint`
`workspace/.mcp.json`, `workspace/user-claude.example.json`	Project servers with `${GITHUB_TOKEN}`; a user-scope example for personal servers
`workspace/src/utils/formatDate.js`	The one-file bug for task T1 (null `shippedAt` crashes the export)
`tasks.json`	T1 one-file bug, T2 47-file migration, T3 notification architecture choice
`simulator.py`, `verify.py`	Driver that runs each step through the Agent SDK; acceptance checks
`cases/UC-*.json`, `casekit.py`	Ten practice-test use cases and the runner that executes and scores them
`hooks/`	PostToolUse hook that normalises Bash output (UC-9)
`workspace/.claude/commands/task.md`, `find-callers.md`	Commands used by UC-23 and UC-37

Guided tour

1. Read the project instructions and the two files they import. Everything in these three files reaches Claude on every call in this folder.

```
cat workspace/CLAUDE.md workspace/docs/standards/testing.md
```

2. Run step 1. Claude quotes the imported testing standard, then reads `Button.test.tsx` and `orders.js` and names the rule files that entered its context. Only `tests.md` (and the React rule) load for the test file; only `api-handlers.md` loads for the handler.

```
uv run simulator.py 1
```

3. Run step 2. The driver copies `workspace/` into a scratch git repo, commits it as `main`, then deletes the `orders:read` check from `getOrder` so there is a real diff. `/review main` executes the project slash command non-interactively, finds the removed authorization check and ends with a `VERDICT:` line; the `codebase-map` skill runs in a forked context with Read, Grep and Glob only and returns a short map of `src/api`.

```
uv run simulator.py 2
```

© 2026 Tertiary Infotech Academy · UEN 201200696W

4. Run step 3. T2 runs with `--permission-mode plan`: Claude reads and plans but changes nothing (the file hash table proves it). T1 runs with `--permission-mode acceptEdits`: Claude fixes `formatDate.js` and adds the regression test the standard demands. Both run in a scratch copy.

```
uv run simulator.py 3
```

5. Run step 4. A named investigation session is created, resumed with `--resume <id>` (same id, prior files remembered) and forked with `--fork-session` (new id, same history).

```
uv run simulator.py 4
```

6. Run step 5. The project `.mcp.json` carries `${GITHUB_TOKEN}` placeholders and no literal secret; the SQLite scratch server sits in the user-scope example because teammates should not inherit it. Nothing is written to your real `~/.claude.json`.

```
uv run simulator.py 5
```

7. Read the evidence. Session ids, verbatim answers, the T2 plan excerpt, the fixed source and the cost of every call are in one file.

```
uv run python -m json.tool evidence/run.json | less
```

8. Run the verifier.

```
uv run verify.py
```

9. Stretch: set `LAB_MODEL=claude-sonnet-5-5`, rerun step 3 and compare the T2 plan and the cost line.

Labs that run on this simulator

Open this simulator in your browser: from the `labs` folder run `uv run serve.py`, then pick Simulator 2. In Claude Code, type `/lab` and the simulator number. A simulator is the system. A lab is one exam situation you run on it. Run any of these from the `labs` folder with `uv run run_lab.py <lab>`; each has its own PDF under its topic folder. An asterisk marks a lab run in class.

Lab	Task	Source	What it proves	Command
2	1.1	Practice Q17	Claude re-asks because the tool_result never came back	`uv run casekit.py --case UC-17`
5*	1.3	Practice Q25	A /parallel-review that can't fan out because Task isn't allowed	`uv run casekit.py --case UC-25`
13	1.5	Practice Q9	Shrink noisy npm test output before Claude reads it	`uv run casekit.py --case UC-9`
16*	1.7	Practice Q5	Two attempts, one analysis: fork the session, checkpoint the files	`uv run casekit.py --case UC-5`
23*	2.4	Task statement	Commit the server, never the token	`uv run simulator.py 5`
27*	3.1	Task statement	Which instructions actually reached Claude?	`uv run simulator.py 1`

© 2026 Tertiary Infotech Academy · UEN 201200696W

Lab	Task	Source	What it proves	Command
28	3.2	Guide Q4	A /review command the whole team gets through git	`uv run simulator.py 2`
31*	3.4	Practice Q8	Replacing the auth layer: nobody agrees on the boundary, so plan	`uv run casekit.py --case UC-8`
32	3.4	Practice Q30	Plan first, then approve into an editing mode	`uv run casekit.py --case UC-30`
33*	3.4	Practice Q34	One guard clause doesn't need a plan	`uv run casekit.py --case UC-34`
40*	4.2	Practice Q23	"Clean this up" means three things: show which one you mean	`uv run casekit.py --case UC-23`
59	5.5	Practice Q33	Acceptance looks great. Two fields don't.	`uv run casekit.py --case UC-33`

Expected evidence

- `evidence/run.json`: per-step records with session ids, outputs, plan-vs-direct decisions, MCP scope analysis and cumulative cost.
- `evidence/cases/UC-<n>.json`: one file per use case with every task's session id, reply, files changed and the scored rubric.
- `evidence/verification.txt`: the twelve workspace checks plus one PASS line per use case, with the model and cost used.

Acceptance checks

- The quoted testing standard comes from an `@import` file, not from `CLAUDE.md` itself.
- `tests.md` loads for `src/components/Button.test.tsx` and not for `src/api/orders.js`.
- `/review` returns `VERDICT: approve` or `VERDICT: request-changes`; `codebase-map` mentions the orders handler.
- Plan mode changed zero files; direct execution changed `formatDate.js` and added its test.
- `--resume` keeps the session id; `--fork-session` returns a different one.
- `.mcp.json` holds placeholders only; the user-scope server is not in the project file.

Exam link

Task statements 3.1 (hierarchy, @import, .claude/rules), 3.2 (commands, skills, context: fork, allowed-tools, argument-hint), 3.3 (path-scoped rules), 3.4 (plan mode vs direct execution), 2.4 (MCP scopes and environment expansion), 1.7 (--resume, fork_session), 3.6 (-p and --output-format json).

© 2026 Tertiary Infotech Academy · UEN 201200696W